# Problem 01: Page status and size report

**Difficulty: Easy**

## TASK

Transform the blocking program into an async program. It should fetch every page concurrently, preserve the result formatting, and still report the final lines in the same order as the input.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url. Labels and URLs contain no spaces.

## OUTPUT FORMAT

Print label status_code byte_count for each input row, preserving input order.

## Example input 1

```
3
python https://www.python.org/
docs https://docs.python.org/3/library/asyncio.html
rfc https://www.rfc-editor.org/rfc/rfc9110.html
```

## Expected output 1

```
python 200 51199
docs 200 109697
rfc 200 357273
```

## Example input 2

```
2
jsonplaceholder https://jsonplaceholder.typicode.com/
httpbin https://httpbin.org/
```

## Expected output 2

```
jsonplaceholder 200 7061
httpbin 200 9593
```

## Synchronous code

```
import sys
import requests

DEFAULT_HEADERS = {
    "User-Agent": "sync-to-async-practice",
    "Accept": (
        "text/html,application/json;q=0.9,"
        "*/*;q=0.8"
    ),
}
REQUEST_TIMEOUT = 10

def build_headers():
    return dict(DEFAULT_HEADERS)

def fetch_page(label, url):
    headers = build_headers()
    response = requests.get(
        url,
        headers=headers,
        timeout=REQUEST_TIMEOUT,
    )
    response.raise_for_status()
    size = len(response.content)
    return label, response.status_code, size

def format_result(result):
    label, status_code, size = result
    return f"{label} {status_code} {size}"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def collect_results(rows):
    results = []
    for label, url in rows:
        results.append(fetch_page(label, url))
    return results

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    results = collect_results(rows)
    for result in results:
        print(format_result(result))

main()
```

# Problem 02: Completion-order page fetches

**Difficulty: Easy**

## TASK

Rewrite the program so it starts all fetches first and prints each result as soon as that fetch finishes. The final program should still include the label and status code for every row. The new solution must use async concurrency instead of the current blocking approach.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url.

## OUTPUT FORMAT

Print DONE label status_code in completion order. If two requests complete together, keep the input order.

### Example input 1

```
3
slow https://httpbin.org/delay/2
fast https://httpbin.org/delay/0
mid https://httpbin.org/delay/1
```

### Expected output 1

```
DONE fast 200
DONE mid 200
DONE slow 200
```

### Example input 2

```
2
later https://httpbin.org/delay/1
now https://httpbin.org/delay/0
```

### Expected output 2

```
DONE now 200
DONE later 200
```

### Synchronous code

```
import sys
import requests

HEADERS = {"User-Agent": "sync-to-async-practice"}
TIMEOUT = 10

def build_request(url):
    return {
        "url": url,
        "headers": HEADERS,
        "timeout": TIMEOUT,
    }

def fetch_status(label, url):
    request_kwargs = build_request(url)
    response = requests.get(**request_kwargs)
    response.raise_for_status()
    return label, response.status_code

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def sequential_download(rows):
    completed = []
    for label, url in rows:
        result = fetch_status(label, url)
        completed.append(result)
    return completed

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    completed = sequential_download(rows)
    for label, status_code in completed:
        print(f"DONE {label} {status_code}")

main()
```

# Problem 03: Bounded JSON record lookup

**Difficulty: Easy-medium**

## TASK

Convert the sequential JSON client into an async client with bounded concurrency. Each record fetch is independent, the parser should stay separate from the HTTP logic, and the final report must preserve input order.

## INPUT FORMAT

Line 1: limit n. Next n lines: record_id. Assume 1 <= limit <= n.

## OUTPUT FORMAT

Print record_id name username for each row, preserving input order.

### Example input 1

```
2 3
1
2
3
```

### Expected output 1

```
1 Leanne Graham Bret
2 Ervin Howell Antonette
3 Clementine Bauch Samantha
```

### Example input 2

```
1 2
3
1
```

### Expected output 2

```
3 Clementine Bauch Samantha
1 Leanne Graham Bret
```

### Synchronous code

```
import sys
import requests

BASE_URL = "https://jsonplaceholder.typicode.com/users/{}"
TIMEOUT = 10

def build_url(record_id):
    return BASE_URL.format(record_id)

def parse_record(record_id, data):
    name = data["name"]
    username = data["username"]
    return f"{record_id} {name} {username}"

def load_record(record_id):
    url = build_url(record_id)
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    data = response.json()
    return parse_record(record_id, data)

def read_record_ids(count):
    record_ids = []
    for _ in range(count):
        record_ids.append(sys.stdin.readline().strip())
    return record_ids

def collect_lines(record_ids):
    results = []
    for record_id in record_ids:
        line = load_record(record_id)
        results.append(line)
    return results

def main():
    limit, n = map(int, sys.stdin.readline().split())
    record_ids = read_record_ids(n)
    results = collect_lines(record_ids)
    for line in results:
        print(line)

main()
```

# Problem 04: Repository metadata report

**Difficulty: Easy-medium**

## TASK

Transform the repository reporter into async code. Each row fetches metadata for one public repository, the JSON-to-output step should remain explicit, and the final report must preserve input order. Make sure the async version performs the independent work concurrently.

## INPUT FORMAT

Line 1: integer n. Next n lines: owner repo.

## OUTPUT FORMAT

Print full_name language default_branch for each input row.

### Example input 1

```
2
python cpython
psf requests
```

### Expected output 1

```
python/cpython Python main
psf/requests Python main
```

### Example input 2

```
1
python cpython
```

### Expected output 2

```
python/cpython Python main
```

### Synchronous code

```
import sys
import requests

API_ROOT = "https://api.github.com/repos/{}/{}"
HEADERS = {
    "User-Agent": "sync-to-async-practice",
    "Accept": "application/vnd.github+json",
}
TIMEOUT = 10

def build_url(owner, repo):
    return API_ROOT.format(owner, repo)

def normalize_field(value):
    if value is None:
        return "None"
    return str(value)

def format_repository(data):
    full_name = normalize_field(data.get("full_name"))
    language = normalize_field(data.get("language"))
    branch = normalize_field(data.get("default_branch"))
    return f"{full_name} {language} {branch}"

def load_repository(owner, repo):
    url = build_url(owner, repo)
    response = requests.get(
        url, headers=HEADERS, timeout=TIMEOUT
    )
    response.raise_for_status()
    data = response.json()
    return format_repository(data)

def read_rows(count):
    rows = []
    for _ in range(count):
        owner, repo = sys.stdin.readline().split()
        rows.append((owner, repo))
    return rows

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    output = []
    for owner, repo in rows:
        output.append(load_repository(owner, repo))
    for line in output:
        print(line)

main()
```

# Problem 11: Bounded HEAD request checker

**Difficulty: Easy-medium**

## TASK

Convert the blocking HEAD checker into an async program with a maximum number of active checks. Every row is independent, the status formatting should remain the same, and the final report must preserve input order.

## INPUT FORMAT

Line 1: limit n. Next n lines: label url. Assume 1 <= limit <= n.

## OUTPUT FORMAT

Print label status_code content_type for each row. If the content type header is missing, print UNKNOWN.

### Example input 1

```
2 3
py https://www.python.org/
json https://httpbin.org/json
robots https://www.python.org/robots.txt
```

### Expected output 1

```
py 200 text/html
json 200 application/json
robots 200 text/plain
```

### Example input 2

```
1 2
home https://example.com/
get https://httpbin.org/get
```

### Expected output 2

```
home 200 text/html
get 200 application/json
```

### Synchronous code

```
import sys
import requests

TIMEOUT = 10
HEADERS = {"User-Agent": "sync-to-async-practice"}

def normalize_type(value):
    if not value:
        return "UNKNOWN"
    return value.split(";")[0]

def check_head(label, url):
    response = requests.head(
        url,
        headers=HEADERS,
        timeout=TIMEOUT,
        allow_redirects=True,
    )
    response.raise_for_status()
    content_type = normalize_type(
        response.headers.get("Content-Type")
    )
    return (
        f"{label} {response.status_code} {content_type}"
    )

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def collect(rows):
    output = []
    for label, url in rows:
        output.append(check_head(label, url))
    return output

def main():
    limit, n = map(int, sys.stdin.readline().split())
    rows = read_rows(n)
    for line in collect(rows):
        print(line)

main()
```

# Problem 05: Package metadata collector

**Difficulty: Medium**

## TASK

Rewrite the package metadata loader so it fetches several package documents concurrently while keeping the final output in input order. Keep the response parsing logic readable and preserve the current reporting format. The new solution should be asynchronous.

## INPUT FORMAT

Line 1: limit n. Next n lines: package_name. Assume 1 <= limit <= n.

## OUTPUT FORMAT

Print package_name canonical_name for each input row.

### Example input 1

```
2 3
requests
aiohttp
fastapi
```

### Expected output 1

```
requests requests
aiohttp aiohttp
fastapi fastapi
```

### Example input 2

```
1 2
uvicorn
pydantic
```

### Expected output 2

```
uvicorn uvicorn
pydantic pydantic
```

### Synchronous code

```
import sys
import requests

BASE_URL = "https://pypi.org/pypi/{}/json"
TIMEOUT = 10

def build_url(package_name):
    return BASE_URL.format(package_name)

def extract_name(data):
    info = data.get("info", {})
    return info.get("name", "UNKNOWN")

def load_package(package_name):
    url = build_url(package_name)
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    data = response.json()
    canonical_name = extract_name(data)
    return f"{package_name} {canonical_name}"

def read_package_names(count):
    package_names = []
    for _ in range(count):
        package_names.append(sys.stdin.readline().strip())
    return package_names

def collect_results(package_names):
    results = []
    for package_name in package_names:
        result = load_package(package_name)
        results.append(result)
    return results

def main():
    limit, n = map(int, sys.stdin.readline().split())
    package_names = read_package_names(n)
    results = collect_results(package_names)
    for line in results:
        print(line)

main()
```

# Problem 06: HTML title scraper

**Difficulty: Medium**

## TASK

Transform the blocking title scraper into an async program. Every page should be fetched concurrently, the title extraction helper should stay intact in spirit, and the final titles should be reported in input order.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url.

## OUTPUT FORMAT

Print label title for each row. Strip surrounding whitespace from the extracted title.

## Example input 1

```
2
asyncio https://en.wikipedia.org/wiki/Asynchronous_I/
  O
http https://en.wikipedia.org/wiki/HTTP
```

## Expected output 1

```
asyncio Asynchronous I/O - Wikipedia
http HTTP - Wikipedia
```

## Example input 2

```
1
python https://en.wikipedia.org/wiki/Python_(programm
  ing_language)
```

## Expected output 2

```
python Python (programming language) - Wikipedia
```

## Synchronous code

```
import re
import sys
import requests

TITLE_RE = re.compile(
    r"<title>(.*?)</title>", re.I | re.S
)
TIMEOUT = 10

def normalize_title(text):
    return " ".join(text.split())

def extract_title(html_text):
    match = TITLE_RE.search(html_text)
    if not match:
        return "NO_TITLE"
    title = match.group(1)
    return normalize_title(title)

def fetch_title(label, url):
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    title = extract_title(response.text)
    return f"{label} {title}"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    results = []
    for label, url in rows:
        results.append(fetch_title(label, url))
    for line in results:
        print(line)

main()
```

# Problem 07: Per-request timeout report

**Difficulty: Medium**

## TASK

Convert the timeout checker into async code. A timeout for one request must not prevent the other requests from completing and being reported, and the final output order should remain unchanged. Make sure the async version performs the independent work concurrently.

## INPUT FORMAT

Line 1: timeout_s n. Next n lines: label url. timeout_s is a floating-point number.

## OUTPUT FORMAT

Print label OK for successful responses and label TIMEOUT for timed-out requests, preserving input order.

### Example input 1

```
1.0 3
fast https://httpbin.org/delay/0
slow https://httpbin.org/delay/2
json https://httpbin.org/json
```

### Expected output 1

```
fast OK
slow TIMEOUT
json OK
```

### Example input 2

```
0.5 2
now https://httpbin.org/get
later https://httpbin.org/delay/1
```

### Expected output 2

```
now OK
later TIMEOUT
```

### Synchronous code

```
import sys
import requests

DEFAULT_HEADERS = {"User-Agent": "sync-to-async-practice"}

def build_request(url, timeout_s):
    return {
        "url": url,
        "headers": DEFAULT_HEADERS,
        "timeout": timeout_s,
    }

def run_check(label, url, timeout_s):
    try:
        request_kwargs = build_request(url, timeout_s)
        response = requests.get(**request_kwargs)
        response.raise_for_status()
        return f"{label} OK"
    except requests.Timeout:
        return f"{label} TIMEOUT"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def main():
    timeout_s, n = sys.stdin.readline().split()
    timeout_s = float(timeout_s)
    n = int(n)
    rows = read_rows(n)
    lines = []
    for label, url in rows:
        lines.append(run_check(label, url, timeout_s))
    for line in lines:
        print(line)

main()
```

# Problem 12: JSON field extractor

**Difficulty: Medium**

## TASK

Rewrite the JSON field extractor as async code. Fetch all documents concurrently, keep the field extraction helper separate, and preserve input-order output even when faster documents finish earlier.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url field_name. Field names contain no spaces.

## OUTPUT FORMAT

Print label value. If the field is missing or null, print label MISSING.

### Example input 1

```
3
post https://jsonplaceholder.typicode.com/posts/1 tit
  le
user https://jsonplaceholder.typicode.com/users/1 use
  rname
todo https://jsonplaceholder.typicode.com/todos/1 com
  pleted
```

### Expected output 1

```
post sunt aut facere repellat provident occaecati exc
  epturi optio reprehenderit
user Bret
todo False
```

### Example input 2

```
2
a https://httpbin.org/json slideshow
b https://httpbin.org/json missing
```

### Expected output 2

```
a {...}
b MISSING
```

### Synchronous code

```
import sys
import requests

TIMEOUT = 10

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url, field = sys.stdin.readline().split()
        rows.append((label, url, field))
    return rows

def stringify(value):
    if value is None:
        return "MISSING"
    return str(value)

def extract_field(data, field):
    if field not in data:
        return "MISSING"
    return stringify(data.get(field))

def fetch_json(label, url, field):
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    data = response.json()
    value = extract_field(data, field)
    return f"{label} {value}"

def run_all(rows):
    output = []
    for label, url, field in rows:
        output.append(fetch_json(label, url, field))
    return output

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    for line in run_all(rows):
        print(line)

main()
```

# Problem 13: Download checksum report

**Difficulty: Medium**

## TASK

Transform the checksum downloader into async code. Multiple files should be downloaded concurrently, the hashing logic should stay deterministic, and the final lines should remain in input order.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url.

## OUTPUT FORMAT

Print label byte_count sha256_prefix, where sha256_prefix is the first 12 hex characters of the SHA-256 digest.

## Example input 1

```
2
robots https://www.python.org/robots.txt
uuid https://httpbin.org/uuid
```

## Expected output 1

```
robots 537 e3a1a4f7c91b
uuid 53 2d7b0c1a5e14
```

## Example input 2

```
1
json https://httpbin.org/json
```

## Expected output 2

```
json 429 5a21d902f26b
```

## Synchronous code

```
import hashlib
import sys
import requests

TIMEOUT = 10

def digest_prefix(raw_bytes):
    digest = hashlib.sha256(raw_bytes).hexdigest()
    return digest[:12]

def download(label, url):
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    raw_bytes = response.content
    size = len(raw_bytes)
    prefix = digest_prefix(raw_bytes)
    return f"{label} {size} {prefix}"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def build_report(rows):
    lines = []
    for label, url in rows:
        lines.append(download(label, url))
    return lines

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    report = build_report(rows)
    for line in report:
        print(line)

main()
```

# Problem 14: CSV row counter

**Difficulty: Medium**

## TASK

Rewrite the CSV row counter so several CSV documents are fetched and parsed asynchronously. Network requests should overlap, but each document should still be parsed into a row count before its line is reported.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url.

## OUTPUT FORMAT

Print label data_row_count for each input row, preserving input order. Do not count the header row.

### Example input 1

```
2
a https://example.com/a.csv
b https://example.com/b.csv
```

### Expected output 1

```
a 12
b 7
```

### Example input 2

```
1
small https://example.com/small.csv
```

### Expected output 2

```
small 3
```

### Synchronous code

```
import csv
import io
import sys
import requests

TIMEOUT = 10

def count_data_rows(csv_text):
    reader = csv.reader(io.StringIO(csv_text))
    rows = list(reader)
    if not rows:
        return 0
    return max(0, len(rows) - 1)

def fetch_csv(label, url):
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    row_count = count_data_rows(response.text)
    return f"{label} {row_count}"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def run(rows):
    lines = []
    for label, url in rows:
        lines.append(fetch_csv(label, url))
    return lines

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    for line in run(rows):
        print(line)

main()
```

# Problem 08: Sequential retry chains

**Difficulty: Medium-hard**

## TASK

Transform the retry script into async code. Different rows should run concurrently, the attempts inside one row must stay sequential, and the reporting format should remain the same.

## INPUT FORMAT

Line 1: max_attempts n. Next n lines: label comma_separated_urls.

## OUTPUT FORMAT

Print label OK attempts status if a non-5xx response is found. Otherwise print label FAIL attempts last_status. Preserve input order.

### Example input 1

```
3 2
primary https://httpbin.org/status/500,https://httpbi
  n.org/status/200
broken https://httpbin.org/status/503,https://httpbin
  .org/status/502
```

### Expected output 1

```
primary OK 2 200
broken FAIL 2 502
```

### Example input 2

```
2 1
teapot https://httpbin.org/status/418
```

### Expected output 2

```
teapot OK 1 418
```

### Synchronous code

```
import sys
import requests

TIMEOUT = 10

def should_stop(status_code):
    return status_code < 500

def split_urls(raw_urls):
    return raw_urls.split(',')

def fetch_once(url):
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code

def fetch_with_retries(label, urls, max_attempts):
    last_status = None
    attempts_used = 0
    for attempt_index, url in enumerate(
        urls[:max_attempts], start=1
    ):
        last_status = fetch_once(url)
        attempts_used = attempt_index
        if should_stop(last_status):
            return (
                f"{label} OK {attempts_used} {last_status}"
            )
    return f"{label} FAIL {attempts_used} {last_status}"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, raw_urls = sys.stdin.readline().split()
        rows.append((label, raw_urls))
    return rows

def main():
    parts = sys.stdin.readline().split()
    max_attempts, n = map(int, parts)
    rows = read_rows(n)
    output = []
    for label, raw_urls in rows:
        urls = split_urls(raw_urls)
        output.append(
            fetch_with_retries(label, urls, max_attempts)
        )
    for line in output:
        print(line)

main()
```

# Problem 09: Legacy downloader bridge

**Difficulty: Medium-hard**

## TASK

Rewrite the program so it can be used safely from async code while keeping the blocking helper unchanged. The final program should still perform the same content check and preserve input-order reporting. Make sure the async version performs the independent work concurrently.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url.

## OUTPUT FORMAT

Print label OK if the downloaded text contains the string User-agent. Otherwise print label MISSING. Preserve input order.

## Example input 1

```
2
python https://www.python.org/robots.txt
rfc https://www.rfc-editor.org/robots.txt
```

## Expected output 1

```
python OK
rfc OK
```

## Example input 2

```
1
python https://www.python.org/robots.txt
```

## Expected output 2

```
python OK
```

## Synchronous code

```
import sys
from urllib.request import Request, urlopen

TIMEOUT = 10
USER_AGENT = "sync-to-async-practice"

def build_request(url):
    return Request(
        url, headers={"User-Agent": USER_AGENT}
    )

def download_text(url):
    request = build_request(url)
    with urlopen(request, timeout=TIMEOUT) as response:
        raw_bytes = response.read()
    return raw_bytes.decode("utf-8", "replace")

def inspect_document(label, url):
    text = download_text(url)
    if "User-agent" in text:
        return f"{label} OK"
    return f"{label} MISSING"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    results = []
    for label, url in rows:
        results.append(inspect_document(label, url))
    for line in results:
        print(line)

main()
```

# Problem 15: Paginated API chains

**Difficulty: Medium-hard**

## TASK

Convert the paginated API loader into async code. Different collections should run concurrently, but pages inside the same collection must be requested sequentially because each page determines whether another page is needed.

## INPUT FORMAT

Line 1: integer n. Next n lines: label base_url page_count. The base_url should accept a page query parameter.

## OUTPUT FORMAT

Print label total_items pages_read for each input row, preserving input order.

### Example input 1

```
2
posts https://jsonplaceholder.typicode.com/posts 3
comments https://jsonplaceholder.typicode.com/comment
  s 2
```

### Expected output 1

```
posts 30 3
comments 20 2
```

### Example input 2

```
1
users https://jsonplaceholder.typicode.com/users 1
```

### Expected output 2

```
users 10 1
```

### Synchronous code

```
import sys
import requests

TIMEOUT = 10
PAGE_SIZE = 10

def build_url(base_url, page_number):
    return (
        f"{base_url}?_page={page_number}"
        f"&_limit={PAGE_SIZE}"
    )

def fetch_page(base_url, page_number):
    url = build_url(base_url, page_number)
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    return response.json()

def load_collection(label, base_url, page_count):
    total_items = 0
    pages_read = 0
    for page_number in range(1, page_count + 1):
        data = fetch_page(base_url, page_number)
        pages_read += 1
        total_items += len(data)
        if not data:
            break
    return f"{label} {total_items} {pages_read}"

def read_rows(count):
    rows = []
    for _ in range(count):
        parts = sys.stdin.readline().split()
        label, base_url, page_count = parts
        rows.append((label, base_url, int(page_count)))
    return rows

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    for label, base_url, page_count in rows:
        print(
            load_collection(label, base_url, page_count)
        )

main()
```

# Problem 16: Per-origin request queues

**Difficulty: Medium-hard**

## TASK

Transform the per-origin checker into async code. Requests for different origins may run concurrently, but requests sharing the same origin must remain sequential and preserve their input order within that origin.

## INPUT FORMAT

Line 1: integer n. Next n lines: origin label url.

## OUTPUT FORMAT

Print label status_code in completion order. If completions tie, use original input order.

### Example input 1

```
4
api a1 https://httpbin.org/delay/1
cdn c1 https://example.com/
api a2 https://httpbin.org/get
docs d1 https://docs.python.org/3/
```

### Expected output 1

```
c1 200
d1 200
a1 200
a2 200
```

### Example input 2

```
3
h1 x https://httpbin.org/get
h1 y https://httpbin.org/get
h2 z https://example.com/
```

### Expected output 2

```
x 200
z 200
y 200
```

### Synchronous code

```
import sys
import requests
from collections import defaultdict

TIMEOUT = 10

def fetch_status(label, url):
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    return label, response.status_code

def read_rows(count):
    rows = []
    for index in range(count):
        origin, label, url = sys.stdin.readline().split()
        rows.append((index, origin, label, url))
    return rows

def group_by_origin(rows):
    groups = defaultdict(list)
    for row in rows:
        _, origin, _, _ = row
        groups[origin].append(row)
    return groups

def run_group(rows):
    results = []
    for _, origin, label, url in rows:
        results.append(fetch_status(label, url))
    return results

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    groups = group_by_origin(rows)
    output = []
    for origin in sorted(groups):
        output.extend(run_group(groups[origin]))
    for label, status_code in output:
        print(f"{label} {status_code}")

main()
```

# Problem 10: Shared webpage cache

**Difficulty: Hard**

## TASK

Transform the program into an async cache. If the same URL appears several times, identical lookups should share one in-flight fetch, different URLs should still proceed independently, and the final fetch count should reflect the deduplicated work. Make sure the async version performs the independent work concurrently.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url. The same URL may appear more than once.

## OUTPUT FORMAT

Print label status_code for each input row in input order. Then print FETCHES distinct_url_count.

### Example input 1

```
4
py1 https://www.python.org/
py2 https://www.python.org/
bin https://httpbin.org/get
py3 https://www.python.org/
```

### Expected output 1

```
py1 200
py2 200
bin 200
py3 200
FETCHES 2
```

### Example input 2

```
3
a https://jsonplaceholder.typicode.com/
b https://jsonplaceholder.typicode.com/
c https://httpbin.org/
```

### Expected output 2

```
a 200
b 200
c 200
FETCHES 2
```

### Synchronous code

```
import sys
import requests

TIMEOUT = 10
fetches = 0
cache = {}

def fetch_status(url):
    global fetches
    fetches += 1
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    return response.status_code

def resolve_status(url):
    if url not in cache:
        cache[url] = fetch_status(url)
    return cache[url]

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def build_output(rows):
    lines = []
    for label, url in rows:
        status_code = resolve_status(url)
        lines.append(f"{label} {status_code}")
    return lines

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    output = build_output(rows)
    for line in output:
        print(line)
    print("FETCHES", fetches)

main()
```

# Problem 17: Mirror race for assets

**Difficulty: Hard**

## TASK

Rewrite the mirror downloader so each asset can try its mirrors concurrently and keep the first successful response. Different assets should also run concurrently. Cancel or ignore slower mirror attempts once a successful mirror is selected.

## INPUT FORMAT

Line 1: integer n. Next n lines: asset_id comma_separated_urls.

## OUTPUT FORMAT

Print asset_id OK status_code chosen_url for success. Print asset_id FAIL if all mirrors fail. Preserve input order across assets.

## Example input 1

```
2
logo https://bad.example/logo.png,https://httpbin.org
  /image/png
json https://httpbin.org/status/500,https://httpbin.o
  rg/json
```

## Expected output 1

```
logo OK 200 https://httpbin.org/image/png
json OK 200 https://httpbin.org/json
```

## Example input 2

```
1
broken https://httpbin.org/status/500,https://httpbin
  .org/status/503
```

## Expected output 2

```
broken FAIL
```

## Synchronous code

```
import sys
import requests

TIMEOUT = 10

def split_urls(raw_urls):
    return raw_urls.split(',')

def fetch_mirror(url):
    try:
        response = requests.get(url, timeout=TIMEOUT)
        if response.status_code >= 500:
            return None
        return response.status_code, url
    except requests.RequestException:
        return None

def fetch_asset(asset_id, raw_urls):
    urls = split_urls(raw_urls)
    for url in urls:
        result = fetch_mirror(url)
        if result is not None:
            status_code, chosen_url = result
            return (
                f"{asset_id} OK {status_code} {chosen_url}"
            )
    return f"{asset_id} FAIL"

def read_rows(count):
    rows = []
    for _ in range(count):
        asset_id, raw_urls = sys.stdin.readline().split()
        rows.append((asset_id, raw_urls))
    return rows

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    output = []
    for asset_id, raw_urls in rows:
        output.append(fetch_asset(asset_id, raw_urls))
    for line in output:
        print(line)

main()
```

# Problem 18: Freshness verifier with retries

**Difficulty: Hard**

## TASK

Transform the freshness verifier into async code. Each resource has a sequence of URLs to try sequentially, but different resources should be checked concurrently. Keep retry decisions local to each resource.

## INPUT FORMAT

Line 1: max_attempts n. Next n lines: label comma_separated_urls required_header.

## OUTPUT FORMAT

Print label OK attempts status if the required header is present. Otherwise print label STALE attempts last_status.

### Example input 1

```
2 2
cache https://httpbin.org/response-headers?ETag=abc E
  Tag
missing https://httpbin.org/status/200,https://httpbi
  n.org/status/204 ETag
```

### Expected output 1

```
cache OK 1 200
missing STALE 2 204
```

### Example input 2

```
3 1
lastmod https://httpbin.org/response-headers?Last-Mod
  ified=x Last-Modified
```

### Expected output 2

```
lastmod OK 1 200
```

### Synchronous code

```
import sys
import requests

TIMEOUT = 10

def has_header(response, header_name):
    return header_name in response.headers

def fetch_once(url):
    return requests.get(url, timeout=TIMEOUT)

def verify_resource(
    label, raw_urls, required_header, max_attempts
):
    urls = raw_urls.split(',')
    last_status = 0
    attempts = 0
    for url in urls[:max_attempts]:
        response = fetch_once(url)
        attempts += 1
        last_status = response.status_code
        if has_header(response, required_header):
            return f"{label} OK {attempts} {last_status}"
    return f"{label} STALE {attempts} {last_status}"

def read_rows(count):
    rows = []
    for _ in range(count):
        parts = sys.stdin.readline().split()
        label, raw_urls, required_header = parts
        rows.append((label, raw_urls, required_header))
    return rows

def main():
    parts = sys.stdin.readline().split()
    max_attempts, n = map(int, parts)
    rows = read_rows(n)
    for label, raw_urls, required_header in rows:
        print(verify_resource(
            label, raw_urls, required_header, max_attempts
        ))

main()
```

# Problem 19: Blocking parser bridge

**Difficulty: Hard**

## TASK

Rewrite the program so the blocking parser can be called from async code without freezing other downloads. Keep parse_numbers unchanged, run independent URLs concurrently, and preserve input-order reporting.

## INPUT FORMAT

Line 1: integer n. Next n lines: label url. Each URL returns plain text containing integers separated by whitespace.

## OUTPUT FORMAT

Print label count total for each input row.

## Example input 1

```
2
a https://example.com/a.txt
b https://example.com/b.txt
```

## Expected output 1

```
a 4 18
b 3 10
```

## Example input 2

```
1
empty https://example.com/empty.txt
```

## Expected output 2

```
empty 0 0
```

## Synchronous code

```
import sys
import requests

TIMEOUT = 10

def parse_numbers(text):
    values = []
    for token in text.split():
        if token.lstrip('-').isdigit():
            values.append(int(token))
    return values

def download_text(url):
    response = requests.get(url, timeout=TIMEOUT)
    response.raise_for_status()
    return response.text

def summarize(label, url):
    text = download_text(url)
    values = parse_numbers(text)
    return f"{label} {len(values)} {sum(values)}"

def read_rows(count):
    rows = []
    for _ in range(count):
        label, url = sys.stdin.readline().split()
        rows.append((label, url))
    return rows

def main():
    n = int(sys.stdin.readline())
    rows = read_rows(n)
    output = []
    for label, url in rows:
        output.append(summarize(label, url))
    for line in output:
        print(line)

main()
```

# Problem 20: Three-stage report pipeline

**Difficulty: Hard**

## TASK

Convert the blocking report pipeline into async code. Downloads should overlap, parsing should be bounded by parse_limit, and stores should be bounded by store_limit. The final report should preserve input order while the stages run concurrently.

## INPUT FORMAT

Line 1: parse_limit store_limit n. Next n lines: report_id source_url.

## OUTPUT FORMAT

Print report_id STORED record_count for each input row. If parsing finds no records, print report_id EMPTY 0.

## Example input 1

```
2 1 3
jan https://example.com/jan.json
feb https://example.com/feb.json
mar https://example.com/mar.json
```

## Expected output 1

```
jan STORED 12
feb EMPTY 0
mar STORED 9
```

## Example input 2

```
1 1 1
q1 https://example.com/q1.json
```

## Expected output 2

```
q1 STORED 30
```

## Synchronous code

```
import sys
import requests

TIMEOUT = 10

def download_report(source_url):
    response = requests.get(source_url, timeout=TIMEOUT)
    response.raise_for_status()
    return response.json()

def parse_records(raw_document):
    records = raw_document.get("records", [])
    clean = []
    for record in records:
        if record.get("active"):
            clean.append(record)
    return clean

def store_records(report_id, records):
    if not records:
        return f"{report_id} EMPTY 0"
    stored_count = len(records)
    return f"{report_id} STORED {stored_count}"

def process_report(report_id, source_url):
    document = download_report(source_url)
    records = parse_records(document)
    return store_records(report_id, records)

def read_rows(count):
    rows = []
    for _ in range(count):
        report_id, source_url = (
            sys.stdin.readline().split()
        )
        rows.append((report_id, source_url))
    return rows

def main():
    parts = sys.stdin.readline().split()
    parse_limit, store_limit, n = map(int, parts)
    rows = read_rows(n)
    output = []
    for report_id, source_url in rows:
        output.append(
            process_report(report_id, source_url)
        )
    for line in output:
        print(line)

main()
```